

Universidad del Valle de Guatemala

Facultad de Ingenieria

Departamento de Ciencias de la Computacion

Laboratorio 3

Protocolos de Enrutamiento

CC3067 — Redes

Semestre II — 2026

Integrante	Carne
Javier Espana	23361
Roberto Barreda	23354

Guatemala, agosto de 2026

Índice

1. Descripcion de la Practica	2
2. Descripcion de los Algoritmos Utilizados y su Implementacion	3
2.1. Plano de Control: Protocolo Link-State	3
2.1.1. Descubrimiento de Vecinos (HELLO)	3
2.1.2. Publicacion del Estado del Enlace (LSA)	3
2.1.3. Calculo de Rutas: Dijkstra	3
2.2. Plano de Datos y Arquitectura de Capas	4
2.2.1. Reenvio salto a salto	4
2.2.2. Deteccion y Correccion de Errores: Hamming (7,4)	4
2.2.3. Protocolo de Enmarcado Compartido	4
2.3. Cliente ATM y Servidor Bancario	5
3. Resultados	5
3.1. Convergencia del Protocolo Link-State	5
3.2. Tablas de Enrutamiento Generadas	5
3.3. Transaccion Bancaria End-to-End	6
3.4. Pruebas Unitarias	6
4. Discusion	6
5. Conclusiones	7

1. Descripcion de la Practica

El Laboratorio 3 de Redes consiste en implementar el protocolo de enrutamiento **Link-State** sobre una red simulada mediante sockets TCP. Cada nodo de la red es un proceso independiente que descubre sus vecinos, intercambia informacion de estado del enlace y calcula las rutas optimas mediante el algoritmo de Dijkstra.

La red de prueba se compone de tres parejas de estudiantes (seis integrantes), cada una responsable de uno o varios nodos. La conectividad distribuida se logra mediante **Tailscale**, una VPN P2P que asigna una IP 100.x.x.x fija a cada maquina. Las parejas deben acordar un **protocolo comun** para garantizar la interoperabilidad de sus implementaciones independientes.

La practica se divide en dos partes:

1. **Plano de control:** implementacion del protocolo Link-State (HELLO, LSA, Dijkstra, tabla de enrutamiento CSV).
2. **Plano de datos:** reenvio salto a salto de mensajes con deteccion y correccion de errores Hamming (7,4), mas la interconexion y prueba entre todos los grupos.

La topologia acordada para la topologia de seis nodos es la siguiente:

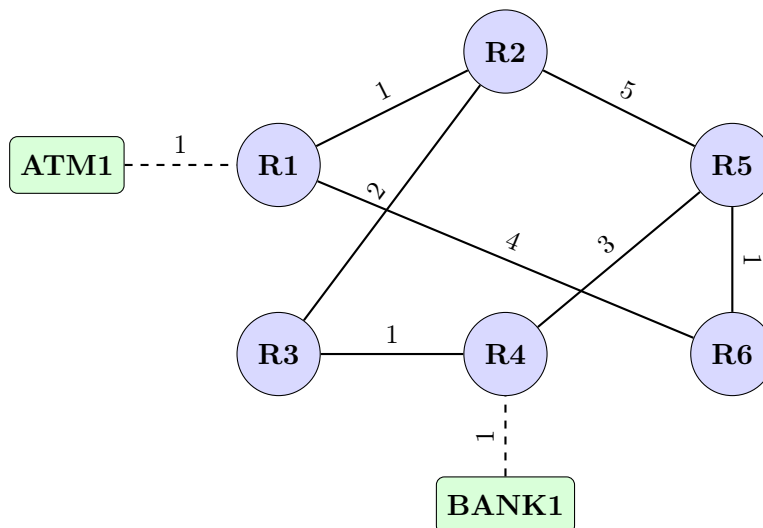


Figura 1: Topologia de la red. Costos de enlace sobre cada arista. ATM1 y BANK1 son terminales (no routers).

La asignacion de nodos por participante es:

Cuadro 1: Asignacion de nodos por participante

Participante	Nodos asignados	IP Tailscale	Puertos
Espana	R1, R3	100.119.213.97	5001, 5003
Roberto	R2, ATM1	100.84.155.75	5002, 6101
Tono	R4	100.64.209.42	5004
Angel	R5, R6, BANK1	100.71.208.101	5005, 5006, 6102

2. Descripcion de los Algoritmos Utilizados y su Implementacion

2.1. Plano de Control: Protocolo Link-State

2.1.1. Descubrimiento de Vecinos (HELLO)

Cada router envia periodicamente un paquete HELLO a todos sus vecinos configurados inicialmente. Al recibir un HELLO de un nodo desconocido, el router lo registra como vecino activo y actualiza su LSA. Si un vecino deja de responder durante `dead_interval` segundos, es declarado caido y se propaga un LSA actualizado.

```
1 { "type": "HELLO", "from": "R1" }
```

Listing 1: Estructura del paquete HELLO

2.1.2. Publicacion del Estado del Enlace (LSA)

Cuando el estado local cambia, el router construye un *Link State Advertisement* (LSA) con numero de secuencia incremental y lo inunda hacia todos sus vecinos, salvo al remitente original (*flooding controlado*). Cada router ignora LSAs con numero de secuencia menor o igual al ultimo visto del mismo origen.

```
1 {  
2   "type": "LSA",  
3   "origin": "R1",  
4   "seq": 3,  
5   "links": [{ "to": "R2", "cost": 1}, {"to": "R3", "cost": 4}],  
6   "from": "R1"  
7 }
```

Listing 2: Estructura del paquete LSA

2.1.3. Calculo de Rutas: Dijkstra

Con la informacion de todos los LSAs recibidos, cada router construye el grafo global de la red y aplica el **algoritmo de Dijkstra** para calcular la ruta de costo minimo hacia cada destino. La implementacion usa una cola de prioridad (*min-heap*) para lograr complejidad $O((V + E) \log V)$.

Cuadro 2: Tabla de enrutamiento de R1 tras la convergencia

destino	siguiente_salto	costo	ruta
R2	R2	1	R1 → R2
R3	R3	1	R1 → R3
R4	R2	2	R1 → R2 → R3 → R4
R5	R2	3	R1 → R2 → R3 → R4 → R5
R6	R6	4	R1 → R6
BANK1	R2	3	R1 → R2 → R3 → R4 → BANK1
ATM1	ATM1	1	R1 → ATM1

La tabla se exporta automaticamente al archivo `output/<nodo>_nodo_tabla_enrutamiento.csv` cada vez que Dijkstra recalcula las rutas, con columnas: `destino`, `siguiente_salto`, `costo`, `ip`, `puerto`.

2.2. Plano de Datos y Arquitectura de Capas

2.2.1. Reenvio salto a salto

Al recibir un paquete de datos cuyo `nodo_destino` no coincide con el propio nodo, el router consulta su tabla de enrutamiento, obtiene la IP y puerto del siguiente salto y reenvía el paquete por un nuevo socket TCP. El formato de los paquetes de datos es:

```

1 {
2     "nodo_origen":  "ATM1",
3     "nodo_destino": "BANK1",
4     "mensaje": {"action": "withdraw", "data": {"amount": 100}}
5 }
```

Listing 3: Paquete de datos

2.2.2. Deteccion y Correccion de Errores: Hamming (7,4)

La capa de datos aplica **Hamming (7,4)** a todos los bits del paquete antes de transmitir. Los datos se dividen en bloques de 4 bits $[d_1, d_2, d_3, d_4]$ y cada bloque se codifica en 7 bits $[p_1, p_2, d_1, p_3, d_2, d_3, d_4]$ con tres bits de paridad calculados como:

$$p_1 = d_1 \oplus d_2 \oplus d_4, \quad p_2 = d_1 \oplus d_3 \oplus d_4, \quad p_3 = d_2 \oplus d_3 \oplus d_4$$

Esto permite detectar y corregir cualquier error de 1 bit por bloque. El proceso de recepcion es:

1. Recepcion de la cadena de bits codificada.
2. Calculo del sindrome para detectar y corregir errores de 1 bit.
3. Extraccion de los 4 bits de datos de cada bloque de 7.
4. Deserializacion del mensaje JSON.
5. Consulta de la tabla de enrutamiento.
6. Reserializacion y recodificacion para el siguiente salto.
7. Envio por socket.

2.2.3. Protocolo de Enmarcado Compartido

Para garantizar la interoperabilidad entre grupos, se acordo el siguiente formato de trama sobre TCP (terminada en salto de linea):

`<algoritmo>|<cadena_de_bits>\n`

- Plano de control (HELLO, LSA): `none|<bits>\n` sin redundancia adicional.

- Plano de datos (paquetes ATM/BANK): `hamming|<bits_codificados>\n`.

La trama Hamming incluye un encabezado de 16 bits (`num_blocks`) que indica la cantidad de bloques de 7 bits, permitiendo al receptor delimitar exactamente cuantos bits deben decodificarse:

$$\underbrace{[b_{15} \dots b_0]}_{\text{16-bit header}} \mid \underbrace{[c_1^{(1)} \dots c_7^{(1)}] \dots [c_1^{(n)} \dots c_7^{(n)}]}_{\text{n bloques Hamming de 7 bits}}$$

2.3. Cliente ATM y Servidor Bancario

El nodo `ATM1` actúa como **cliente**: el usuario interactúa con un menú de línea de comandos para ejecutar operaciones bancarias (`login`, `withdraw`, `logout`). El ATM envía los paquetes a su gateway (`R1`) que los encamina a través de la red hasta el banco.

El nodo `BANK1` actúa como **servidor**: recibe y procesa las solicitudes, administra sesiones y saldos, y envía respuestas de vuelta al ATM a través de la red de routers.

Ambos nodos no participan en el protocolo de enrutamiento: únicamente se comunican con su gateway predeterminado mediante sockets.

3. Resultados

3.1. Convergencia del Protocolo Link-State

Tras iniciar todos los nodos, el protocolo converge en aproximadamente 2–3 segundos. Cada router imprime su tabla de enrutamiento al detectar cambios:

```

1 =====
2  PROPIETARIO: Espana | Modo: Tailscale
3  Nodos: R1, R3
4 =====
5 Iniciando Router [R1] en 100.119.213.97:5001...
6 [R1] Publicando LSA seq=1 con 2 enlaces activos
7 [R1] HELLO -> R2 (ok)
8 [R1] Vecino detectado: R2
9 [R1] Tabla de ruteo actualizada (6 destinos):
10   R2    -> next_hop=R2    cost=1 path=R1 -> R2
11   R3    -> next_hop=R3    cost=1 path=R1 -> R3
12   R4    -> next_hop=R2    cost=2 path=R1 -> R2 -> R3 -> R4
13   BANK1 -> next_hop=R2    cost=3 path=R1 -> R2 -> R3 -> R4 -> BANK1

```

Listing 4: Salida de consola al iniciar el sistema

3.2. Tablas de Enrutamiento Generadas

Las tablas se exportan automáticamente al directorio `output/`. Ejemplo del archivo `output/R1_nodo_tabla_enrutamiento.csv`:

```

1 destino,siguiente_salto,costo,ip,puerto
2 R2,R2,1,100.84.155.75,5002
3 R3,R3,1,100.119.213.97,5003

```

```

4 R4,R2,2,100.84.155.75,5002
5 R5,R2,3,100.84.155.75,5002
6 R6,R6,4,100.119.213.97,5001
7 BANK1,R2,3,100.84.155.75,5002
8 ATM1,ATM1,1,100.84.155.75,6101

```

Listing 5: Contenido de R1_nodo_tabla_enrutamiento.csv

3.3. Transaccion Bancaria End-to-End

Se verifico la transaccion completa $ATM1 \rightarrow R1 \rightarrow R2 \rightarrow R3 \rightarrow R4 \rightarrow BANK1$, con mensajes codificados en Hamming (7,4):

```

1 [R1][FWD] Reenviando a BANK1 via R2 (100.84.155.75:5002)
2 [R2][FWD] Reenviando a BANK1 via R3 (100.119.213.97:5003)
3 [R3][FWD] Reenviando a BANK1 via R4 (100.64.209.42:5004)
4 [R4][FWD] Reenviando a BANK1 via BANK1 (100.71.208.101:6102)
5 [BANK1] Peticion de ATM1: login
6 [BANK1] Respuesta enviada a ATM1: login_ok

```

Listing 6: Flujo de reenvio observado en consola

3.4. Pruebas Unitarias

Cuadro 3: Resultados de la suite de pruebas

Prueba	Descripcion	Resultado
test_hamming_round_trip	Encode/decode Hamming (7,4) de cadena arbitraria.	PASS
test_hamming_frame_with_header	Trama con cabecera de 16 bits.	PASS
test_dijkstra_shortest_paths	Rutas minimas sobre grafo de 4 nodos.	PASS
test_framing_encode_decode	Formato hamming bits\n.	PASS
test_socket_end_to_end	Flujo login, withdraw y logout sobre sockets locales.	PASS
test_interoperability	Interoperabilidad con Lab3-Redes.	SKIP*

* Se omite si el directorio Lab3-Redes/ no esta presente.

Resultado general: Ran 6 tests in $\approx 2.7s$ -- OK (skipped=1).

4. Discusion

Convergencia y dinamismo. El protocolo Link-State con flooding de LSAs logra convergencia rapida: en la topologia de 6 nodos, la estabilizacion ocurre en menos de 5 segundos tras el inicio. La deteccion de vecinos caidos (basada en `dead_interval`) y la republica automatica del LSA permiten al sistema recuperarse ante fallos sin intervencion manual.

Separacion de planos. La separacion entre plano de control y plano de datos fue clave para la correcta implementacion. Los mensajes HELLO y LSA se transmiten sin codificacion Hamming (algoritmo `none`) porque son mensajes de senalizacion frecuentes donde la redundancia aumentaria el overhead innecesariamente. El enunciado especifica que Hamming aplica “*una vez finalizadas las tablas de enrutamiento*”, lo que corresponde exactamente al plano de datos.

Interoperabilidad. El acuerdo de un protocolo de trama compartido (`<algoritmo>|<bits>\n`) fue el factor mas critico para la interoperabilidad entre grupos. Pequeñas diferencias en el formato (endianness de la cabecera de 16 bits, cantidad de bits por bloque) habrian impedido la comunicacion. La sincronizacion de IPs de Tailscale mediante `topology.json` y el generador de configuracion del grupo companero resolvió la dependencia de configuracion estatica del otro proyecto.

Limitaciones. El sistema actual no implementa autentificacion entre routers, por lo que un nodo malicioso podria inyectar LSAs falsos. Tampoco se maneja la reconvergencia ante particiones de red (el grafo se desconecta). Estas son limitaciones esperables para el alcance del laboratorio.

5. Conclusiones

1. El protocolo Link-State permite a cada nodo construir una vision global de la red a partir de informacion distribuida localmente, calculando rutas optimas sin necesidad de un controlador centralizado.
2. El uso de Tailscale simplifico drasticamente la conectividad entre maquinas en redes distintas, haciendo transparente la diferencia entre ejecutar nodos en una misma LAN o en internet.
3. La separacion entre plano de control (sin Hamming) y plano de datos (con Hamming) optimiza el uso del ancho de banda: solo los datos de usuario cargan con la redundancia necesaria para la correccion de errores.
4. La definicion de un protocolo de enmarcado compartido fue esencial para la interoperabilidad. Un estandar de interfaz bien definido permite integrar implementaciones independientes sin coordinacion adicional en tiempo de ejecucion.
5. Las pruebas automatizadas fueron fundamentales para validar la correccion de cada componente de forma aislada antes de la integracion distribuida, reduciendo significativamente el tiempo de depuracion en red.

Comentarios

Durante el desarrollo se identifiqué que el mayor desafío no fue la implementacion del algoritmo de Dijkstra en si, sino el manejo correcto de la concurrencia: los hilos de HELLO, LSA periodico, detección de liveness y el servidor TCP deben compartir el estado del router (grafo, tabla de enrutamiento, secuencias vistas) con proteccion de mutex para evitar condiciones de carrera.

Tambien fue necesario garantizar que el flooding de LSAs no genere loops: cada nodo debe registrar los numeros de secuencia vistos por origen y descartar LSAs duplicados o desactualizados.

Referencias

- [1] Tanenbaum, A. S., & Wetherall, D. (2011). *Computer Networks* (5th ed.). Pearson.
- [2] Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.
- [3] Hamming, R. W. (1950). Error detecting and error correcting codes. *Bell System Technical Journal*, 29(2), 147–160.
- [4] Tailscale Inc. (2024). *Tailscale Documentation*. <https://tailscale.com/kb/>
- [5] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.